REVIEW 1 • WEEK 7

Restaurant Table Reservation & Food Service Management System

Requirements, Scope & Conceptual Database Design

Vyshnavi.S
Roll No: 25WU0102277
Section: AI ML Leopards 2025–2029
Woxsen University

- Problem & Objectives
- Scope & User Roles
- ER Model
- Initial Relational Schema

Problem Statement & Objectives

The Problem

Restaurants coordinate table booking, order capture, kitchen processing, billing and payment simultaneously. Manual, paper-based methods commonly lead to:

- Overlapping / double-booked table reservations
- Lost or miscommunicated kitchen orders
- Delayed service and inaccurate billing
- No consolidated view of sales, discounts or waiter performance

A relational database eliminates these conflicts through structured data, enforced constraints and coordinated workflows.

Project Objectives

1. **Model the Domain**

   Identify entities, attributes, relationships and user roles across the full reservation-to-payment lifecycle.

2. **Normalize to 3NF**

   Convert the ER model into a relational schema, removing redundancy and update anomalies.

3. **Enforce Business Rules**

   Apply PK/FK, UNIQUE, NOT NULL, CHECK and DEFAULT constraints for data integrity.

4. **Enable Reporting**

   Support joins, aggregates, nested queries and views for operational reports.

5. **Build a Front End**

   Deliver a Java/Python app for reservations, orders, billing and reporting.

Scope & User Roles

In Scope

- ✓ Dining areas & table inventory
- ✓ Reservations & seating
- ✓ Menu, orders & order items
- ✓ Kitchen ticket workflow
- ✓ Billing, discounts & payments

Out of Scope (Phase 1)

- ✗ Online/3rd-party delivery integration
- ✗ Inventory & supplier procurement
- ✗ Loyalty / CRM marketing modules
- ✗ Multi-branch chain consolidation

User Roles



Receptionist

Creates & manages reservations, assigns tables, checks availability



Waiter

Takes orders, updates serving status, assists billing



Kitchen Staff

Views kitchen tickets, updates preparation status



Cashier / Manager

Applies discounts, closes bills, records payments, views reports

Functional Requirements

1

Reservation Management

Book, modify and cancel reservations; prevent overlapping bookings for the same table; validate guest count against table capacity.

2

Seating & Table Status

Track table state (available / reserved / occupied / cleaning) across dining areas in real time.

3

Order Entry

Capture order items per table with quantity, price and special notes; support multiple rounds per visit.

4

Kitchen Ticket Workflow

Route order items to the kitchen; track preparation status and flag delays.

5

Billing & Discounts

Generate itemized bills, apply authorized discounts, and lock bills once closed (immutable).

6

Payment Processing

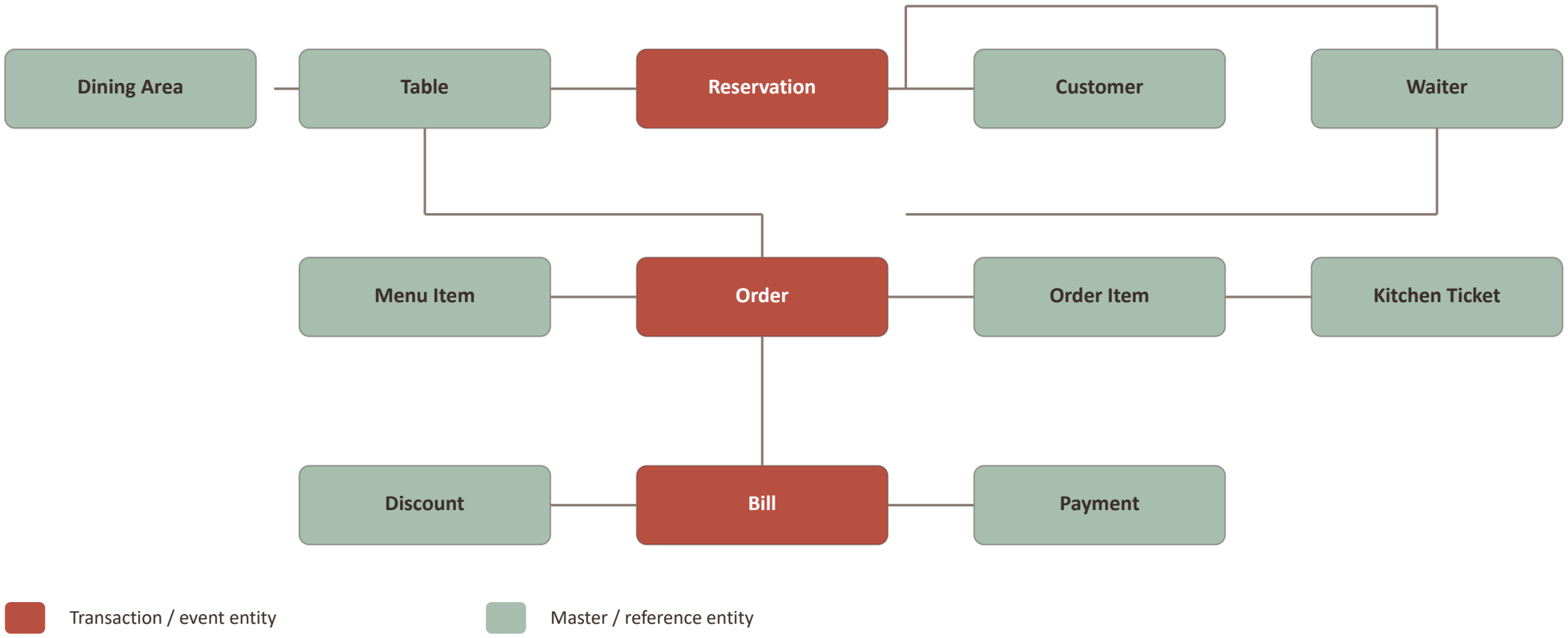
Record one or more payments per bill across supported payment methods.

7

Reporting

Produce views for table turnover, waiter performance, item sales, kitchen delays and revenue.

Conceptual Design — Entity-Relationship Overview



Every relationship carries cardinality (1:1, 1:N, M:N) and participation rules, detailed in the full ER diagram deliverable.

Initial Relational Schema

DiningArea

- PK area_id
- area_name
- description

RestaurantTable

- PK table_id
- FK area_id
- table_no
- capacity
- status

Customer

- PK customer_id
- full_name
- phone
- email

Reservation

- PK reservation_id
- FK table_id
- FK customer_id
- res_date
- start_time
- guest_count
- status

Waiter

- PK waiter_id
- full_name
- shift

MenuItem

- PK item_id
- item_name
- category
- price

KitchenTicket

- PK ticket_id
- FK order_item_id
- sent_time
- status

Order

- PK order_id
- FK table_id
- FK waiter_id
- order_time
- status

OrderItem

- PK order_item_id
- FK order_id
- FK item_id
- quantity
- unit_price

Bill

- PK bill_id
- FK order_id
- FK discount_id
- total_amt
- is_closed

Discount

- PK discount_id
- code
- pct_value
- authorized_by

Payment

- PK payment_id
- FK bill_id
- amount
- method

Normalized to 3NF — every non-key attribute depends on the whole key and nothing but the key.

Business Rules & Next Steps

Key Business Rules (enforced via constraints)

- No overlapping reservations for the same table (date/time range check)
- Guest count must not exceed table capacity (CHECK constraint)
- Order quantity must be positive (CHECK quantity > 0)
- Discounts require an authorized_by reference (NOT NULL FK)
- Closed bills are immutable — no further edits once is_closed = TRUE

Path to Review 2

1. **Finalize 3NF schema**

   Complete all table definitions with keys & constraints

2. **Implement in RDBMS**

   Build schema in MySQL / PostgreSQL / Oracle

3. **Populate sample data**

   Realistic rows across all 12 tables

4. **Write SQL queries**

   Joins, aggregates, nested queries, views, reports

Thank you — Questions welcome